

Formulario — Analisi degli Algoritmi

Algoritmi e Strutture Dati · Notazioni asintotiche, Ordinamento, Ricorrenze

Notazioni Asintotiche

O grande — limite superiore

$$f(n) = O(g(n)) \iff \exists c > 0, n_0 \geq 0 : \\ \forall n > n_0, f(n) \leq c \cdot g(n)$$

f cresce **al più** come g .

Ω grande — limite inferiore

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 \geq 0 : \\ \forall n > n_0, f(n) \geq c \cdot g(n)$$

f cresce **almeno** come g . f non è minore di $c \cdot g$.

Θ grande — limite esatto

$$f(n) = \Theta(g(n)) \iff \exists c_1, c_2 > 0, n_0 \geq 0 : \\ \forall n > n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

f cresce **esattamente** come g . $\Theta = O \cap \Omega$.

o piccolo — strettamente inferiore

$$f(n) = o(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

f cresce **strettamente meno** di g .

ω piccolo — strettamente superiore

$$f(n) = \omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = +\infty$$

f cresce **strettamente più** di g .

Proprietà delle notazioni

Proprietà	O	Ω	Θ
Riflessiva	✓	✓	✓
Transitiva	✓	✓	✓
Simmetrica	×	×	✓

Proprietà	o	ω
Riflessiva	×	×
Transitiva	✓	✓
Simmetrica	×	×

Dimostrazione riflessività Θ : scegli $c_1 = c_2 = 1 \Rightarrow f(n) \leq f(n) \leq f(n)$. ✓

Dimostrazione simmetria Θ :

Se $c_1 g \leq f \leq c_2 g$, dividendo: $\frac{1}{c_2} f \leq g \leq \frac{1}{c_1} f$. ✓

o piccolo non è riflessivo: $\lim_{f \rightarrow f} \frac{f}{f} = 1 \neq 0$.

Simmetria trasposta: $f = O(g) \iff g = \Omega(f)$

Proprietà utili

Regola della somma (max):

$$O(f(n) + g(n)) = O(\max\{f(n), g(n)\})$$

Perché: $\max\{f, g\} \leq f + g \leq 2 \cdot \max\{f, g\}$

Base dei logaritmi: non conta!

$$\log_a n = \frac{\log_b n}{\log_b a} = c \cdot \log_b n \Rightarrow \Theta(\log_a n) = \Theta(\log_b n)$$

Per questo si scrive sempre $O(\log n)$ senza specificare la base.

Dimostrazione $f(n) = 3n^2 + 5n + 1 = \Theta(n^2)$:

Per $n > 4$: $n < \frac{n^2}{4}$ e $1 < \frac{n^2}{4}$, quindi:

$$3n^2 \leq 3n^2 + 5n + 1 \leq 3n^2 + \frac{5n^2}{4} + \frac{n^2}{4} = \frac{9}{2}n^2$$

Quindi $c_1 = 3$, $c_2 = \frac{9}{2}$, $n_0 = 4$.

Gerarchia delle funzioni

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \\ \subset O(n^2) \subset O(n^3) \subset O(2^n)$$

InsertionSort

Pseudocodice iterativo

```
INSERTION_SORT(A, n):  
  for i = 2 to n:  
    key = A[i]  
    j = i - 1  
    while j >= 1 and A[j] > key:  
      A[j+1] = A[j]  
      j = j - 1  
    A[j+1] = key
```

Complessità InsertionSort

Caso	Input	Complessità
Migliore	già ordinato	$\Theta(n)$
Peggior	inverso	$\Theta(n^2)$
Quasi ord. (k cost.)	dist. $\leq k$	$\Theta(n)$
Quasi ord. ($k = O(n/2)$)	dist. $\leq n/2$	$\Theta(n^2)$

Perché quasi ordinato con k costante è $\Theta(n)$?

Ogni elemento si sposta al più k posizioni:

$$T(n) = \sum_{i=1}^n O(k) = O(nk) \xrightarrow{k=\text{cost.}} \Theta(n)$$

Pseudocodice ricorsivo

```

INSERTION_SORT_RIC(A, n):
  if n <= 1: return
  INSERTION_SORT_RIC(A, n-1)
  INSERT(A, n)

INSERT(A, n):
  key = A[n]; j = n-1
  while j>=1 and A[j]>key:
    A[j+1]=A[j]; j=j-1
  A[j+1] = key

```

Ricorrenza:

$$T(n) = \begin{cases} O(1) & n \leq 1 \\ T(n-1) + O(n) & n > 1 \end{cases}$$

Soluzione: $T(n) = O(1) + c \sum_{i=2}^n i = O(n^2)$

Nota: la versione ricorsiva **non migliora** la complessità.

Sommatorie fondamentali

Formula di Gauss:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$$

$$\sum_{i=2}^n i = \frac{n(n+1)}{2} - 1 = \Theta(n^2)$$

Serie geometrica con $r \neq 1$:

$$\sum_{i=0}^{k-1} r^i = \frac{1-r^k}{1-r}$$

Se $r < 1$: $\sum_{i=0}^{\infty} r^i = \frac{1}{1-r} = O(1)$

Se $r > 1$: $\sum_{i=0}^{k-1} r^i = \Theta(r^k)$

Serie geometrica pesata con $r < 1$:

$$\sum_{i=0}^{\infty} i \cdot r^i = \frac{r}{(1-r)^2} = O(1)$$

Master Theorem

Per $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, $a \geq 1$, $b > 1$:

Caso 1: $f(n) = O(n^{\log_b a - \varepsilon})$
 $\Rightarrow T(n) = \Theta(n^{\log_b a})$ (domina la ricorsione)

Caso 2: $f(n) = \Theta(n^{\log_b a})$
 $\Rightarrow T(n) = \Theta(n^{\log_b a} \log n)$ (pareggio)

Caso 2 esteso: $f(n) = \Theta(n^{\log_b a} \log^k n)$
 $\Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Caso 3: $f(n) = \Omega(n^{\log_b a + \varepsilon})$
 $\Rightarrow T(n) = \Theta(f(n))$ (domina il costo esterno)

Applicazioni del Master Theorem

Ricorrenza	$n^{\log_b a}$	$f(n)$	Caso	Soluzione
$4T(n/2) + 3n^2$	n^2	n^2	2	$\Theta(n^2 \log n)$
$2T(n/5) + n \log n$	$n^{0.43}$	$n \log n$	3	$\Theta(n \log n)$
$3T(n/2) + n^2$	$n^{1.58}$	n^2	3	$\Theta(n^2)$
$2T(n/2) + 1$	n	1	1	$\Theta(n)$
$2T(n/2) + n^3$	n	n^3	3	$\Theta(n^3)$
$9T(n/3) + n$	n^2	n	1	$\Theta(n^2)$
$16T(n/4) + n^2 \log^3 n$	n^2	$n^2 \log^3 n$	2+	$\Theta(n^2 \log^4 n)$
$5T(n/3) + n^2$	$n^{1.46}$	n^2	3	$\Theta(n^2)$

Metodo dello Srotolamento

4 passi obbligatori:

1. **ESPANDI** — sostituisci T dentro sé stessa 2-3 volte
2. **PATTERN** — scrivi la formula generale al passo k
3. **CASO BASE** — trova k tale che $\frac{n}{b^k} = 1 \Rightarrow k = \log_b n$
4. **SOMMA** — calcola il totale e trova il termine dominante

Costo al livello i : $a^i \cdot f\left(\frac{n}{b^i}\right)$

Costo livello i	Tipo somma	Caso MT
decresce	geometrica $r < 1 \Rightarrow O(1)$	Caso 3
costante	$\sum 1 = k = \log n$	Caso 2
cresce	geometrica $r > 1 \Rightarrow \Theta(r^k)$	Caso 1

Esempio — $T(n) = 2T(n/2) + n$:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn \xrightarrow{k=\log_2 n} n \cdot O(1) + n \log n = \Theta(n \log n)$$

Esempio — $T(n) = 4T(n/2) + n$:

Costo livello i : $4^i \cdot \frac{n}{2^i} = n \cdot 2^i$ (cresce) $\Rightarrow$ Caso 1

$$T(n) = n^2 \cdot O(1) + n \cdot \Theta(n) = \Theta(n^2)$$

Ricerca in matrice ordinata

Proprietà matrice: ogni riga ordinata sx→dx, ogni colonna ordinata alto→basso.

Algoritmo	Idea	Complessità
Naïve	scansione completa	$\Theta(n^2)$
Bin. per righe	bin. search su ogni riga	$\Theta(n \log n)$
Bin. 2D	bin. search su righe e col.	$O(\log^2 n)$
Ottimale	angolo alto-destra	$O(n)$

Algoritmo ottimale — pseudocodice

```
RICERCA_OTTIMALE(A, n, k):  
  i = 1; j = n  
  while i <= n and j >= 1:  
    if A[i][j] == k: return (i,j)  
    elif A[i][j] > k: j = j - 1  
    else: i = i + 1  
  return NOT_FOUND
```

Correttezza:

- $A[i][j] > k$: k non può stare in colonna j (tutti sotto sono $\geq A[i][j]$)
- $A[i][j] < k$: k non può stare in riga i (tutti a sx sono $\leq A[i][j]$)

Complessità con potenziale:

Definiamo $\Phi = (n - i) + (j - 1)$.

$\Phi_{inizio} = 2n - 2$. Ad ogni passo Φ decresce di 1.
 $\Rightarrow$ al più $2n - 2$ iterazioni $\Rightarrow O(n)$.

Problemi e complessità

Contare occorrenze pari (ricorsivo):

$$T(n) = T(n - 1) + O(1) \Rightarrow \Theta(n)$$

Invariante algoritmo ricorsivo (schema generale):

1. **Caso base:** verificare per $n = 0$ o $n = 1$
2. **Passo induttivo:** assumere vero per $n - 1$, dimostrare per n
3. **Terminazione:** il parametro decresce $\Rightarrow$ raggiunge il caso base

Formule rapide

$$3n^2 + 2 = O(n^2) \text{ con } c = 4, n_0 = 2 \quad (2 \leq n^2 \text{ per } n \geq 2)$$

$$5n + 6 = O(n^2) \text{ con } c = 1, n_0 = 6 \quad ((n-6)(n+1) \geq 0)$$

$$\log_a n = \frac{\log_b n}{\log_b a} \Rightarrow \Theta(\log_a n) = \Theta(\log_b n)$$

$$f = \Theta(g) \iff f = O(g) \text{ e } f = \Omega(g)$$

$$f = o(g) \Rightarrow \lim \frac{g}{f} = +\infty \Rightarrow g = \omega(f)$$